Peer-to-Peer End-to-End Encrypted Chat Application

SEGPRD - Project 2

2024/2025

Class M1B

João Veiga (1201082)
Rui Barbosa (1211106)

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Abstract

Due to the content of the Final Report of the High-Level Group (HLG) on access to data for effective law enforcement and the refusal to disclose the members of the group, there is concern that private communications could be at risk in the European Union. We propose a Peer-to-Peer and End-to-End Encrypted Chat Application, with no central server or authority subject to legislation on implementation of backdoors in encryption algorithms. The proposed application is simple and uses standard and publicly available libraries and algorithms, allowing for replication by any technically knowledgeable user. We present a Proof of Concept (POC) of said application as a web application, where a user can connect to another and exchange messages encrypted by custom protocol using known secure key-exchange methods with quantum security. The POC was publicly deployed and tested, with issues highlighted and mitigation proposed or implemented. The application is demonstrated to provide encrypted messaging between users, using P2P communications.

Acronyms

E2EE End-to-End Encryption.

ECDH Elyptic Curve Diffie-Hellman.

HLG High-Level Group.

MITM Man-in-the-middle.

P2P Point-to-Point.

POC Proof of Concept.

RCS Rich Communication Services.

Contents

Introduction

The report produced by the European Commission's High-Level Group (HLG) on Access to Data for Effective Law Enforcement ("Going Dark") [1] proposes access by Law Enforcement to all devices and communications, with the inclusion of encryption backdoors, expansion of data retention and device monitoring [2]. These measures reduce the overall security of all information systems and introduce the risk of mass surveillance, either by government entities or malicious actors with access to the proposed backdoors.

The HLG has been operating in a non-transparent manner, with no knowledge of its members. Dr. Patrick Breyer, former Member of the European Parliament, requested access to all documents related to the group's work, including the members list [3]. The European Commission's response included the members list, censoring the names of all members [4] [5] [6] [7].

Given the ironic need for privacy of the group members, but overall disregard for the privacy of the European Citizens and their communications, we propose a simple Point-to-Point (P2P) Chat Application, featuring End-to-End Encryption (E2EE), with no central server and focus on privacy and data protection. The application should be easily reproducible, giving the users the possibility of developing their own version, compatible with others, forgoing the existence of a central responsible entity.

Current Landscape

2.1 Existing solutions

For E2EE messaging applications, multiple solutions exist and are widely used. The most popular is WhatsApp, owned by Meta (formerly known as Facebook), with billions of downloads on app stores [8]. Given its ownership by Meta, a for-profit company known for selling user data to advertisers, it should not be trusted for absolute privacy but just enough privacy in day-to-day conversations.

Signal, developers of the Signal Protocol, is owned by the Signal Foundation, a non-profit organisation, with a focus on protecting free expression and secure communications [9]. Given the current state of uncertainty surrounding the Government of the United States of America, we cannot ensure that the privacy and security provided will not change.

Telegram is a messaging application that supports client-server encryption for most conversations and E2EE for "Secret Chats". This architecture can provide less privacy, as the servers hold both the ciphertext and the keys to decrypt it [10]. As the owners of the company, Nikolai Durov and Pavel Durov, originate from the former Soviet Union, current-day Russia, government intrusion on their servers and application cannot be discarded, given the known disregard for the privacy of the current Russian Government.

Rich Communication Services (RCS) is a communication protocol developed by the GSM Association, implemented on most SMS messaging apps [11]. Some applications implementing RCS added their own encryption on top of the protocol, with E2EE being officially added on March 2025, but not yet implemented[1] [12]. Until E2EE is fully implemented by the protocol, and proven secure by researchers, the service should not be fully trusted for secure communications.

2.2 Technology Analysis

2.2.1 Signal Protocol

The Signal Protocol is the encryption protocol used by Signal, WhatsApp and Google Messages (over RCS). It provides encryption for voice and text messaging, combining multiple algorithms to provide secure key exchange/derivation and encryption. The protocol has been analysed by researchers from multiple universities, concluding Signal is cryptographically secure, with no major design flaws detected [13].

2.2.2 MTProto

The MTProto protocol, used by Telegram, provides encryption for client-to-client and client-to-server communications. The protocol was analysed in December 2020 by researchers from the University of Udine, in Italy, proving the protocol secure. The team

[1] As of June 6, 2025.

also stated that, because both ciphertext and plaintext go through Telegram's servers and the server is responsible for choosing the encryption parameters, the server cannot be considered secure. If users do not verify the fingerprints of their shared keys, Man-in-the-middle (MITM) attacks could be possible, rendering the encryption useless [14].

Project Definition

This project aims to develop a web-based Point-to-Point (P2P) messaging application with End-to-End Encryption (E2EE), focused on protecting user privacy and eliminating reliance on centralized servers. The application provides a secure and transparent alternative to centralized communication platforms, ensuring that only the intended recipients can access the exchanged messages. It will be open-source and easily reproducible, empowering users to build and customize their own compatible versions, promoting digital sovereignty and control over their communication infrastructure.

3.1 Required Resources

To determine which resources and technologies to use, we first must define a set of requirements for the application.

- **Ensure secure communication through End-to-End Encryption (E2EE)**, preventing unauthorized access to message contents.
- **Eliminate dependency on central servers** by implementing a fully P2P architecture.
- **Promote user privacy and data sovereignty** by avoiding user data collection and offering open-source, reproducible builds.
- **Offer a simple and intuitive user experience** to encourage adoption by non-technical users.
- **Future proofing** by using post-quantum encryption algorithms.

3.1.1 Technologies

Frontend Development

- HTML5/CSS3 and modern JavaScript
- DOM-based interface elements (<textarea>, <input>, etc.)
- TypeScript: for development-time and security.

Encryption & Security

- Web Crypto API: for ECDH key generation, HKDF-based key derivation, and AES-GCM symmetric encryption.
- crystals-kyber-js: JavaScript implementation of ML-KEM (Kyber) post-quantum encryption.

- HKDF and AES-GCM: used to derive and securely exchange session keys.
- OWASP Secure Coding Practices: to avoid common vulnerabilities in client-side cryptographic applications.

3.1.2 Infrastructure

The application will be hosted using GitHub's Pages feature, on a custom domain. The domain will have HTTPS enabled and enforced. Due to the lack of central servers, no other infrastructure is needed.

Design and Implementation

4.1 Architecture Overview

The application will provide multi-layer encryption, with the implementation of a custom encryption algorithm encapsulated by the standard DTLS protocol.

4.1.1 Point-to-Point Communication

The basis of the project is the P2P communication between clients. For web-based applications, the best option is WebRTC, a web communication library developed by Google for video-communication [15]. The library also supports text-based communications, that we will use.

For development, the PeerJS library was selected, a wrapper for WebRTC [16], as it simplifies both development work and usability, as the only requirements are the User ID of the peer. PeerJS already provides an encryption layer to the communications, by using the DTLS protocol, a UDP implementation of TLS.

4.1.2 End-to-End Encryption

Because of the requirements stating we should provide post-quantum encryption, we cannot rely on PeerJS's use of DTLS for secure cryptography. The decision was made to use Hybrid Key Exchange, combining Elyptic Curve Diffie-Hellman (ECDH) and ML-KEM (also known as Kyber).

The Diffie-Hellman Key Exchange is know for providing a secure method of generating a symmetric key over a public channel, without sharing the key itself [17]. The key is derived using the user's private key and the peer's public key, resulting on the same key for both users. Although the key exchange is considered secure, the encryption can be broken by a quantum computer running Shor's Algorithm [18], thus not meeting the requirement for quantum security.

For quantum security, the ML-KEM (Kyber) algorithm was selected, a NIST Standard for post-quantum key exchange. Similar to ECDH, the algorithm allows the establishment of a shared secret between two parties over a public channel. ML-KEM provides three parameter sets, ML-KEM-512 (NIST security level 1), ML-KEM-768 (NIST security level 3) and ML-KEM-1024 (NIST security level 5) [19].

After both algorithms are used for deriving the respective shared keys, the keys are combined using HKDF to obtain a singular secure session key [20]. The messages are encrypted using AES-256 combined with the Galois/Counter Mode (GCM) for confirming message integrity.

With this, our encryption technique provides secure key exchange and derivation, quantum security and encrypted message integrity validation. The hybrid approach was chosen

because ML-KEM is a recent standard, still being evaluated for it's security. The communication can be considered secure, even if one of the two algorithms is broken, providing redundancy to the encryption.

4.2 Development Environment

The development of the Proof of Concept was done using the WebStorm IDE, developed by JetBrains for Web-Application development. A GitHub repository was used for sharing the project code, automated builds and deployment of the application.

4.3 Implementation steps

The first prototype developed used only PeerJS's built-in encryption using DTLS, providing a simple P2P chat between two users. PeerJS requires the usage of a synchronization server for the connection. The communication does not travel through the server, per the standard, only allowing for the identification of peers using text IDs. The application was set to use the Public PeerJS Server, but a dedicated server could be setup for this purpose.

After testing the P2P functionality, the encryption was added to application. Because both ECDH and ML-KEM require exchange of information for derivation of the respective keys, upon connecting to a new user the key exchange starts. The user that received the connection request starts the key exchange (initiator), by sending it's public keys for both algorithms. The other peer receives the initiators keys and replies with it's own keys. Once both peers have their respective peer's key both generate the ECDH shared secret. The initiator then starts the ML-KEM encapsulation, generating a ciphertext, to be sent to the peer, and the shared key. The other peer, decapsulates the ciphertext, obtaining the shared key. Once both peers have derived the shared keys, they can be combined using HKDF, deriving a new key to be used by AES-GCM for message encryption and decryption. The process was represented visually in the diagram of Figure 4.1.

The application has been deployed to a public domain (`https://p2p.ruifpb.pt`), with enforced HTTPS connection.

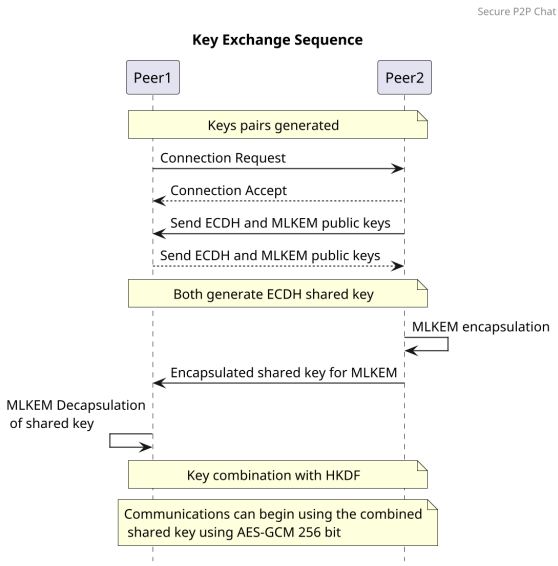

Figure 4.1: Key Exchange Sequence Diagram

Issues Encountered and Mitigations

- **WebRTC blocked:** WebRTC, being a P2P protocol, is often blocked by organizational networks (in our case ISEP/P.Porto) or even Internet Providers. As a mitigation, if the network allows, a VPN allowing P2P connections can be used to circumvent this blocking. Because it's a block on the protocol, there is nothing that can be done, in the application, to fix the issue.
- **Sharing of GUIDs:** By default, PeerJS uses 128 bit IDs for it's users. Due to it's length, they are hard to share with other users to establish communication. To fix the issue, the application generates a simpler username, that can be changed to anything else by the user, allowing for easier sharing.
- **Peer authentication:** Due to the custom usernames, a user can easily impersonate another one. The application does not implement any mechanism for peer authentication, as that would require user authentication with a central server to authorize the use of a specific username, or permanent public keys for sharing and authentication. The issue was not fixed in the Proof of Concept developed, but a mechanism for uploading/using public GPG keys could be implemented for authentication. At the moment, the user is responsible for verifying the peer it is talking to is the intended person.
- **New user can destroy connection:** If two users are in a session and a new user connects to one of the previous users, the previous connection is destroyed to establish the new connection. This issue is not mitigated as the application is simply a Proof of Concept (POC) for testing the encryption and communication strategy. It can easily be fixed by implementing a confirmation box for received connections.

Demonstration

6.1 UI Layout

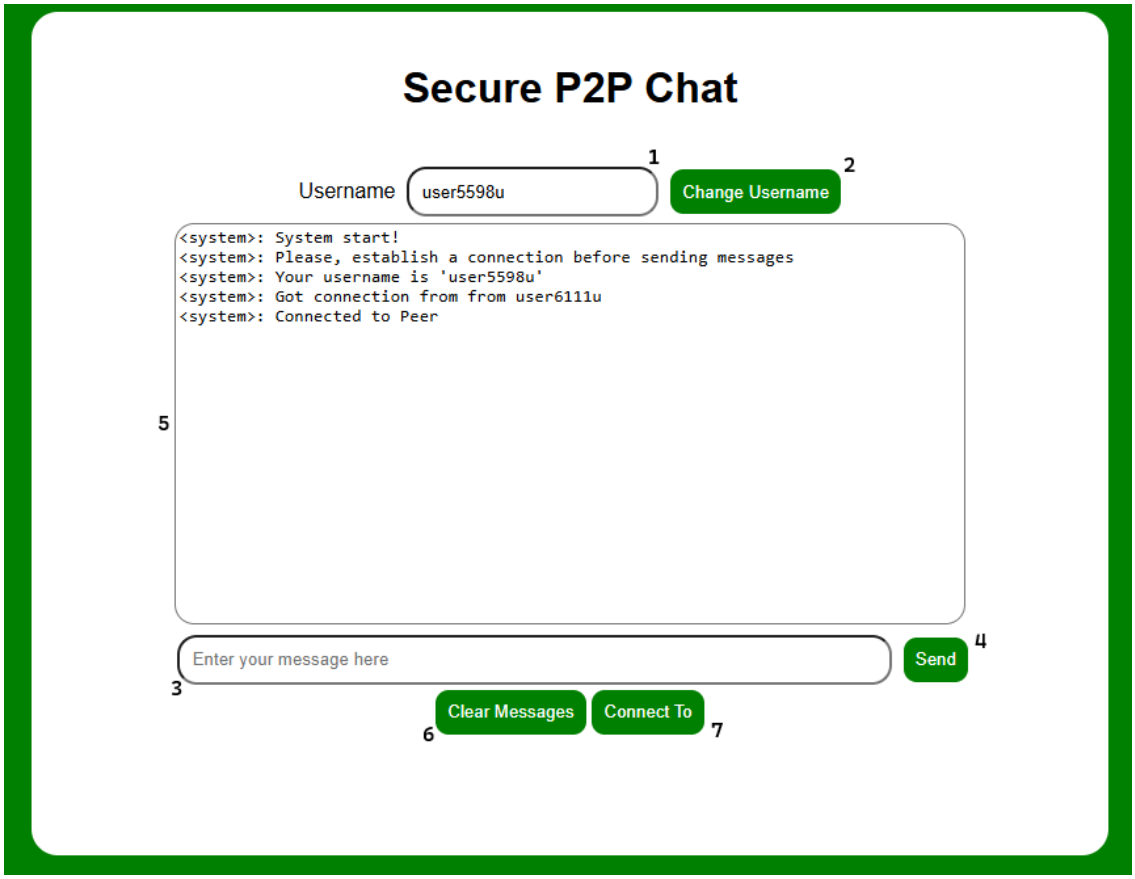

Figure 6.1: Demo application UI

1. **Username Textbox** – Allows to see their username and change it.
2. **Username Button** – Allows the user to change their current username.
3. **Message Input Field** – Textbox where users can type messages to send.
4. **Send Button** – Sends the typed message to the connected peer.
5. **Chat Window** – Displays system messages and the chat history between peers.
6. **Clear Messages Button** – Clears all messages in the chat window.
7. **Connect To Button** – Establishes a connection to another user/peer.

6.2 Demonstration

Figure 6.2: Demo of the application usage

The application starts by assigning the user a temporary username, as shown in the chat window log. Users can personalize this by entering a new name in the Username Textbox **(1)** and clicking the Change Username Button **(2)**.

To begin a conversation, the user must first connect to a peer by clicking the Connect To Button **(7)** and inserting the other user's username in the pop-up. Once connected, messages can be typed in the Message Input Field **(3)** and sent using the Send Button (4). All messages and connection updates appear in the Chat Window **(5)**.

Users may also clear the conversation log at any time using the Clear Messages Button **(6)**.

This simple UI ensures users can intuitively engage in peer-to-peer communication, with essential controls clearly visible and easy to use.

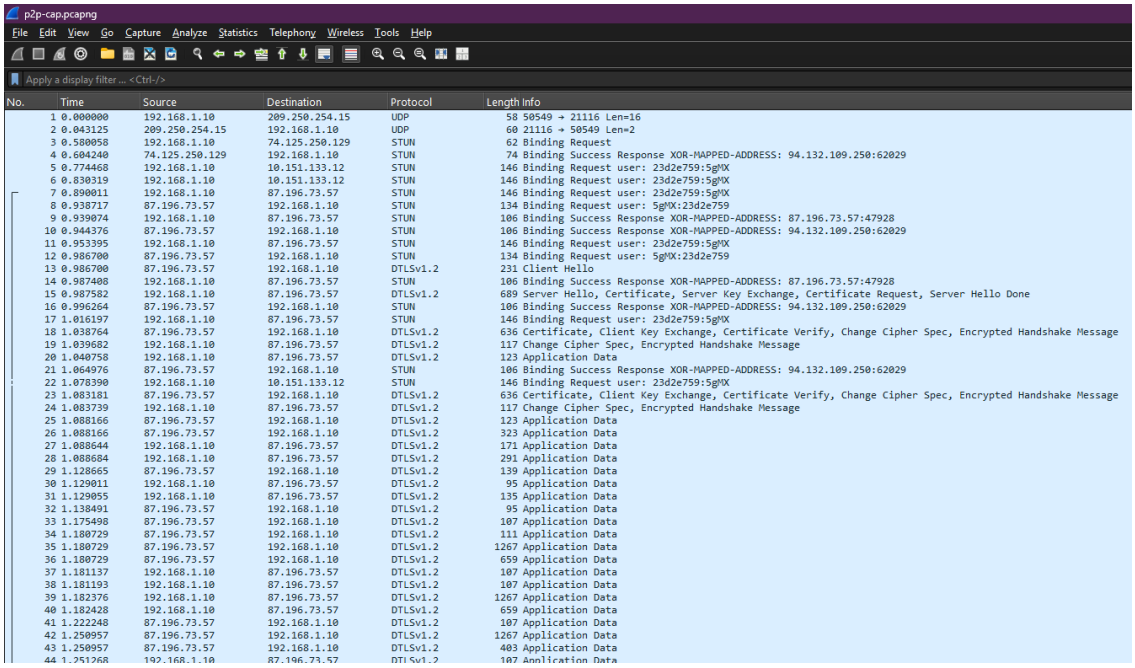

Figure 6.3: Capture of packets from the application

The figure 6.3 shows a Wireshark capture of the network traffic generated during a typical session of the peer-to-peer chat application. Initially, multiple STUN (Session Traversal Utilities for NAT) messages are exchanged to discover the public-facing IP addresses and ports of both peers, enabling NAT traversal.

Following STUN negotiation, a DTLS 1.2 (Datagram Transport Layer Security) handshake is initiated to establish a secure communication channel. This includes the exchange of certificates and key information required for encryption.

Once the handshake is successfully completed, the peers begin exchanging encrypted messages over the DTLS channel. These packets are marked as "Application Data" in the capture, confirming that secure end-to-end communication has been established between the users.

This capture verifies the application's ability to set up a secure peer-to-peer connection using industry-standard protocols.

Bibliography

[1] European Commission, *High-Level Group (HLG) on access to data for effective law enforcement*, [Accessed 03-06-2025]. [Online]. Available: https://home-affairs.ec.europa.eu/networks/high-level-group-hlg-access-data-effective-law-enforcement_en.

[2] High-Level Group, *Concluding report of the High-Level Group on access to data for effective law enforcement*, [Accessed on 2025-06-03]. [Online]. Available: https://home-affairs.ec.europa.eu/document/download/4802e306-c364-4154-835b-e986a9a49281_en?filename=Concluding%20Report%20of%20the%20HLG%20on%20access%20to%20data%20for%20effective%20law%20enforcement_en.pdf.

[3] P. Breyer, *June and November Meetings of the HLEG on access to data for effective law enforcement*, [Accessed on 2025-06-03]. [Online]. Available: https://fragdenstaat.de/anfrage/june-and-november-meetings-of-the-hleg-on-access-to-data-for-effective-law-enforcement/#nachricht-848493.

[4] European Commission, *Participants list, 1st Plenary Meeting*, [Accessed on 2025-06-03]. [Online]. Available: https://fragdenstaat.de/anfrage/june-and-november-meetings-of-the-hleg-on-access-to-data-for-effective-law-enforcement/848493/anhang/document17-participantlistfirstplenary.pdf.

[5] European Commission, *Participants list, Working Group 1*, [Accessed on 2025-06-03]. [Online]. Available: https://fragdenstaat.de/anfrage/june-and-november-meetings-of-the-hleg-on-access-to-data-for-effective-law-enforcement/848493/anhang/document18-participantlistfirstmeetingofwg1.pdf.

[6] European Commission, *Participants list, Working Group 2*, [Accessed on 2025-06-03]. [Online]. Available: https://fragdenstaat.de/anfrage/june-and-november-meetings-of-the-hleg-on-access-to-data-for-effective-law-enforcement/848493/anhang/document19-participantlistfirstmeetingofwg2.pdf.

[7] European Commission, *Participants list, Working Group 3*, [Accessed on 2025-06-03]. [Online]. Available: https://fragdenstaat.de/anfrage/june-and-november-meetings-of-the-hleg-on-access-to-data-for-effective-law-enforcement/848493/anhang/document20-participantlistfirstmeetingofwg3.pdf.

[8] Meta, *WhatsApp | Secure and Reliable Free Private Messaging and Calling — whatsapp.com*, [Accessed 05-06-2025]. [Online]. Available: https://www.whatsapp.com.

[9] Signal Foundation, *Signal Messenger: Speak Freely — signal.org*, [Accessed 05-06-2025]. [Online]. Available: https://signal.org.

[10] Telegram Messeger, *Telegram – a new era of messaging — telegram.org*, [Accessed 06-06-2025]. [Online]. Available: https://telegram.org.

[11] GSMA, *Rich Communication Services — gsma.com*, [Accessed 06-06-2025]. [Online]. Available: https://www.gsma.com/solutions-and-impact/technologies/networks/rcs/.

[12] T. V. Pelt, *RCS Encryption: A Leap Towards Secure and Interoperable Messaging — gsma.com*, [Accessed 06-06-2025]. [Online]. Available: https://www.gsma.com/newsroom/article/rcs-encryption-a-leap-towards-secure-and-interoperable-messaging/.

[13] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, *A formal security analysis of the signal messaging protocol*, Cryptology ePrint Archive, Paper 2016/1013, 2016. [Online]. Available: https://eprint.iacr.org/2016/1013.

[14] M. Miculan and N. Vitacolonna, "Automated verification of telegram's mtproto 2.0 in the symbolic model," *Computers & Security*, vol. 126, p. 103 072, 2023, issn: 0167-4048. doi: https://doi.org/10.1016/j.cose.2022.103072. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404822004643.

[15] *WebRTC — webrtc.org*, [Accessed 07-06-2025]. [Online]. Available: https://webrtc.org.

[16] *PeerJS - Simple peer-to-peer with WebRTC — peerjs.com*, [Accessed 07-06-2025]. [Online]. Available: https://peerjs.com/.

[17] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. doi: 10.1109/TIT.1976.1055638.

[18] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, 2017. arXiv: 1706.06752 [quant-ph]. [Online]. Available: https://arxiv.org/abs/1706.06752.

[19] N. I. of Standards and Technology, *Federal Information Processing Standard (FIPS) 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard — csrc.nist.gov*, [Accessed 07-06-2025]. doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final.

[20] H. Krawczyk, *Cryptographic extraction and key derivation: The HKDF scheme*, Cryptology ePrint Archive, Paper 2010/264, 2010. [Online]. Available: https://eprint.iacr.org/2010/264.